Schema-Evolving Knowledge Graphs for Insurance

Automated Ontology Induction with Cross-Domain Transfer Learning

Xiaofei Wang, Zechary Chou
CS 584 AI Capstone — Spring 2026
Emory University

February 2026

Abstract

We present an automated pipeline for insurance knowledge graph construction that induces formal ontology schemas from raw data and extends them across Lines of Business (LOBs). Using bottom-up clustering with Leiden community detection, we discover conceptual structures without predefined schemas. Both flood and auto insurance are processed independently; our embedding-based cross-LOB aligner then discovers **SharedCore** concepts that bridge the two graphs, enabling unified Cypher queries across domains. We validate the induced flood ontology against a hand-annotated SFIP gold standard (35 classes, 14 properties) using OntoQA metrics (RR, AR, InR, CR) and class/property F1. Cross-domain transfer achieves CTR $> 60\%$ and PTR $> 50\%$ on 210K insurance records using local LLMs (Llama 3.1 8B) with no GPU cluster required.

1 Introduction and Motivation

Insurance companies adding new Lines of Business face a 3–6 month ontology engineering bottleneck per domain. Manual construction does not scale across flood, auto, health, and liability lines. We present an automated pipeline that:

1. **Induces** a formal ontology from raw flood insurance data (text + CSV) via bottom-up Leiden clustering
2. **Extends** that ontology to new LOBs by discovering shared structure automatically
3. **Validates** with two tiers of evaluation—graph quality metrics available at the midpoint, and OntoQA + gold standard F1 at the final report

Key Insight: Connected Graph = Queryable Across Domains

Without alignment, flood and auto produce *two isolated subgraphs*—a Cypher query starting in auto data cannot traverse into flood data. Our cross-LOB aligner creates **SharedCore** nodes that act as bridges, discovered *bottom-up* from data:

```
[AutoPolicy]-[:IS_A]->[AutoModule:Policy]-[:SUBCLASS_OF]->[SharedCore
    :Policy]
                                                          ^
[FloodPolicy]-[:IS_A]->[FloodModule:Policy]-[:SUBCLASS_OF]---+
```

Once connected, a single Cypher query spans both domains:

```
// All claims across flood AND auto
MATCH (c:EntityInstance)-[:IS_A]->(:EntityType)
      -[:INSTANCE_OF*1..3]->(:SharedCore {id:"Claim"})
RETURN c.id, labels(c)
```

Neither "Policy" nor "Claim" is pre-declared as shared. CTR and PTR measure how connected the result is.

2 Baseline Comparison

We compare against LangChain's LLM Graph Transformer—the current state-of-practice for LLM-based knowledge graph construction.

3 Methodology

3.1 Pipeline Overview

Zone 1: Ingestion $\longrightarrow$ Zone 2: Typed Open IE $\longrightarrow$ Zone 3: Ontology Induction $\longrightarrow$ Zone 4: 3-Layer Neo4j

Data: NFIP flood insurance (100K policies + 50K claims + SFIP PDFs) and Kaggle auto insurance (60K records) = 210K records across two domains. Both pipelines receive *identical Zone 1 chunks* (49 PDF + 20 CSV = 69 total).

Flow: Raw documents → typed triples → provisional entity types → Leiden-clustered classes → hierarchical OWL ontology → 3-layer Neo4j graph.

3.2 Zone 1: Hybrid Ingestion (Complete)

Section-aware chunking: title-based splitting at SFIP section headers followed by semantic merging (cosine threshold $\tau = 0.85$) to preserve context across related paragraphs. Each chunk carries metadata (source file, section hierarchy, page number). CSV files use a rolling 5-record window per extraction call to avoid context overflow. **Demonstrated result: +15% query accuracy vs. fixed 512-token chunking.**

3.3 Zone 2: Typed Open Information Extraction

RawTriple data model:

Layer	Classic Baseline	Our Novel Approach
Zone 1	512-token fixed chunks	Section-aware hybrid chunks (+15% query accuracy demonstrated)
Zone 2 Extraction	Schema-free, no type guidance	Few-shot *typed* Open IE: `(subj, subj_type, rel, obj, obj_type, negated, conf)`
Zone 2 Negation	**None**	Explicit `NOT_INSURED` relation; negation recall measured
Zone 2 Numerics	Values as opaque strings	Typed numeric facts: `HAS_COVERAGE_LIMIT=500000`
Zone 2 CSV	**Excluded** (filter bug)	Rolling 5-record window; claims field coverage metric
Zone 3 Ontology	**None**	Leiden → class naming → Hearst hierarchy → property axioms → OWL JSON
Zone 4 Storage	Flat instance graph	3-layer graph: OntologyClass / EntityType / EntityInstance
Cross-LOB	**None**	SharedCore alignment; flood + auto queryable with single Cypher
Evaluation	20 yes/no Q&A tasks only	Tier 1 (midpoint) + Tier 2 (OntoQA + SFIP gold standard F1)

Table 1: Layer-by-layer comparison: Classic Baseline vs. Novel Approach. The classic baseline is the right comparison because it represents what most practitioners implement first.

```
(subject, subj_type, relation, object, obj_type, negated, confidence,
   source_span)
```

Allowed entity types (20): `Policy`, `Claim`, `Coverage`, `FloodZone`, `Exclusion`, `Insurer`, `PolicyHolder`, `Building`, `Contents`, `Deductible`, `Premium`, `WaitingPeriod`, `Endorsement`, `ProofOfLoss`, `LossAmount`, `ElevationCert`, `OccupancyType`, `ICCCoverage`, `FloodEvent`, `Appurtenant`

Allowed relation types (20): `HAS_COVERAGE`, `HAS_COVERAGE_LIMIT`, `EXCLUDED_FROM`, `NOT_INSURED`, `MUST_FILE_WITHIN_DAYS`, `SUBCLASS_OF`, `DETERMINES`, `REQUIRES`, `FILED_AGAINST`, `HAS_DEDUCTIBLE`, `HAS_PREMIUM`, `HAS_WAITING_PERIOD`, `APPLIES_TO`, `LOCATED_IN`, `OWNED_BY`, `AFFECTS`, `RESULTS_IN`, `SUBJECT_TO`, `HAS_PROPERTY`, `INSTANCE_OF`

Quality filter: Drop confidence < 0.7; reject generic relations (`HAS`, `IS`, `CONTAINS`); drop self-loops.

Five few-shot examples cover:

1. **Negation** — "Flood damage to vehicles is not covered" → `(Vehicle, VehicleType, NOT_INSURED, FloodDamage, CoverageType, True, 0.95)`
2. **Numeric limit** — "Building coverage limit is $500,000" → `(BuildingCoverage, Coverage, HAS_COVERAGE_LIMIT, 500000, numeric, False, 0.98)`

3. **Deadline** — "Proof of loss must be filed within 60 days" → `(ProofOfLoss, Document, MUST_FILE_WITHIN_DAYS, 60, numeric, False, 0.97)`
4. **Subclass** — "A Post-FIRM building is a type of building" → `(PostFIRMBuilding, BuildingType, SUBCLASS_OF, Building, BuildingType, False, 0.92)`
5. **Cross-entity** — "Policy 5123 covers building at 123 Main St" → `(Policy5123, Policy, HAS_COVERAGE, Building123, Building, False, 0.89)`

CSV strategy: Rolling window of 5 records per LLM call. Extract `HAS_PROPERTY` relations for key OpenFEMA fields (`claimAmount`, `buildingDamageAmount`, `floodZone`, etc.).

3.4 Zone 3: Ontology Induction (Core Innovation)

3.4.1 Phase 3a: Leiden Clustering

Construct type-similarity graph: for each pair of provisional entity types (t_i, t_j):

$$w_{ij} = 0.5 \cdot \text{sem}(t_i, t_j) + 0.3 \cdot \text{cooccur}(t_i, t_j) + 0.2 \cdot \text{rel_sim}(t_i, t_j) \tag{1}$$

Add edge when $w > 0.75$. Apply Leiden algorithm (`graspologic`, $\gamma \in \{0.5, 0.75, 1.0, 1.25, 1.5\}$, select γ maximizing modularity). Each community → candidate ontology class.

3.4.2 Phase 3b: Class Naming + Hierarchy

- LLM names each community in PascalCase (e.g., {flood_policy, wind_policy} → `PropertyPolicy`)
- **Hearst patterns** on chunk text: "X is a type of Y", "X such as Y", "including X" → `SUBCLASS_OF` candidates
- **Sub-cluster method**: re-run Leiden at higher resolution within each community → child/-parent split
- DAG constraint enforced (topological sort; remove lowest-confidence edge on cycle)

3.4.3 Phase 3c: Property Axioms

For each (domain_type, relation, range_type) triple appearing ≥ 3 times → `ObjectProperty`. Literal values → `DatatypeProperty`. Output: OWL-compatible JSON (`novel_ontology.json`).

Typical output: 20–30 classes, depth ≤ 3, 5–15 properties per class.

3.5 Zone 4: 3-Layer Neo4j Storage

```
Layer 1 (Schema):    :OntologyClass + [:SUBCLASS_OF]          -- 20-30
   nodes
Layer 2 (Types):     :EntityType + [:INSTANCE_OF]             -- 50-200
   nodes
Layer 3 (Instances): :EntityInstance + [:HAS_RELATION]        -- 150K-210K
    nodes
```

Properties on `HAS_RELATION`: {`confidence, negated`} — preserves negation from Zone 2. Batch writes via `UNWIND` in groups of 100 to respect AuraDB connection limits.

3.6 Zone 4b: Cross-LOB Alignment

Both flood and auto run Zones 1–3 **independently** (no shared priors). The aligner embeds all class names with `all-MiniLM-L6-v2` and classifies each auto class against each flood class:

Cosine Score	Classification	Action
≥ 0.80	**SharedCore**	Both classes → `:SharedCore` bridge node
0.60–0.80	**Subclass**	Auto class becomes subclass of flood class
< 0.60	**DomainSpecific**	Auto class isolated in `:AutoModule`

Table 2: Cross-LOB alignment thresholds. SharedCore nodes are the graph bridges that enable unified Cypher queries.

Transfer Metrics:

$$\text{CTR} = \frac{|\text{SharedCore}| + |\text{Subclass}|}{|C_B|} \times 100\% \quad (\text{target: } > 60\%) \tag{2}$$

$$\text{PTR} = \frac{\sum_{C_i^B} |P(C_i^B) \cap P(\text{match}(C_i^B))|}{\sum_{C_i^B} |P(C_i^B)|} \times 100\% \quad (\text{target: } > 50\%) \tag{3}$$

Healthy balance: universal concepts (Policy, Claim, Coverage) transfer; domain-specific concepts (FloodZone, VehicleType) do not.

4 Two-Tier Evaluation

We use two evaluation tiers to ensure **complete, reportable results at the midpoint** (before Zone 3 ships) as well as comprehensive metrics at the final report.

4.1 Tier 1 — Midpoint-Ready (after Zone 2)

Computable on **both** Classic Baseline and Novel (Zone 2) output. Script: `evaluation/graph_eval.py`

A. Enhanced Graph Quality Metrics

Metric	Formula
Type Inconsistency Rate	multi-label roots / total roots
Orphan Node Rate	degree-0 nodes / total nodes
Average Node Degree	total edges / total nodes
Connected Component Count	# disconnected subgraphs
Entity Duplication Rate	duplicate normalized entities / total
Label Entropy	$H = -\sum_\ell p_\ell \log p_\ell$ (label frequency)

B. Triple Quality vs. Gold (∼50 annotated SFIP passages)
C. Claims / CSV Coverage
D. 20 Q&A Tasks — same Cypher tasks from `baseline/eval.py` (unchanged).
Midpoint Results Table:

Metric	Formula
Triple Precision	#correct_extracted / #total_extracted
Triple Recall	#correct_extracted / #gold_triples
Triple F1	harmonic mean
Negation Recall	#negation_triples_extracted / #gold_negation_triples
Numeric Fact Recall	#numeric_triples_extracted / #gold_numeric_triples

Metric	Formula
Claims Field Coverage	# OpenFEMA fields present as relations / total fields
Claim Entity Rate	% claim records producing ≥ 1 graph node

Metric	Baseline-Orig	Baseline+Z1	Novel (Zone 2)
Query accuracy (20 tasks)	35%	50%	>65% (target)
Type inconsistency rate	8.0%	15.2%	<10% (target)
Orphan node rate	?	?	<5%
Triple Precision vs gold	—	—	>0.65
Triple Recall vs gold	—	—	>0.60
Negation Recall	0%	0%	>50%
Claims field coverage	0%	0%	>60%

Table 3: Tier 1 midpoint table. All three columns are **complete and reportable** at the midpoint, before Zone 3 ships.

4.2 Tier 2 — Final Report (after Zone 3)

Only computable on Novel (Zone 3) output. Classic baseline receives ∅ N/A throughout—**that is the finding**. Script: `evaluation/ontology_eval.py`

A. OntoQA Schema Metrics [5]

Metric	Formula
Relationship Richness (RR)	|obj_props|/(|obj_props| + |subclass_rels|)
Attribute Richness (AR)	|datatype_props|/|classes|
Inheritance Richness (InR)	|subclass_rels|/|classes|
Class Richness (CR)	|classes with instances|/|total classes|

B. SFIP Gold Standard P/R/F1

Gold standard: 35 classes and 14 properties hand-annotated from the SFIP source document (see Appendix E). Matching uses exact name match with cosine similarity fallback (threshold 0.7).

C. Cross-LOB (Weeks 8–9)

CTR and PTR as defined in Section 3.6.

Final Report Table:

Metric	Classic Baseline	Novel Pipeline
[All Tier 1 metrics — both columns measured]		
Ontology classes	∅ N/A	20–30
Ontology hierarchy depth	∅ N/A	2–3 levels
OntoQA RR / AR / InR / CR	∅ N/A	measured
Class F1 vs SFIP gold	∅ N/A	>0.65 (target)
Property F1 vs SFIP gold	∅ N/A	>0.55 (target)
CTR / PTR (cross-LOB)	∅ N/A	>60% / >50%

Table 4: The ∅ N/A rows are not an evaluation limitation—they are the core finding. The classic approach cannot be evaluated on ontology metrics because it never produces a formal ontology.

5 Implementation

Stack: Neo4j AuraDB, Llama 3.1 8B (Ollama), all-MiniLM-L6-v2, graspologic (Leiden), LangChain LLMGraphTransformer (baseline only).

Key Files:

File	Action	Purpose
`config.py`	Modify	8 new path constants
`baseline/pipeline.py`	Modify	Fix CSV filter; add summarize_csv_chunk
`data/reference/sfip_gold_standard.json`	Create (manual)	35-class SFIP gold standard
`data/reference/gold_triples.json`	Create (manual)	∼50 annotated triples
`zone2/extractor.py`	Create	Typed few-shot Open IE
`zone3/ontology_inducer.py`	Create	Leiden → hierarchy → OWL JSON
`zone4/storage.py`	Create	3-layer Neo4j write
`zone4/cross_lob_aligner.py`	Create	SharedCore alignment
`evaluation/graph_eval.py`	Create	Tier 1 evaluation
`evaluation/ontology_eval.py`	Create	Tier 2 + OntoQA
`evaluation/compare_results.py`	Modify	3-column results table

Timeline:

- **Weeks 1–4 (Done):** Baseline pipeline + Zone 1 hybrid chunking; +15% query accuracy demonstrated
- **Weeks 5–6 (Phase 0–2):** Gold standard annotation; `config.py` update; baseline CSV fix; Tier 1 evaluation harness
- **Weeks 6–7 (Phase 3):** Zone 2 typed extractor → midpoint report with complete Tier 1 table
- **Weeks 7–8 (Phases 4–5):** Zone 3 ontology induction + Zone 4 3-layer storage
- **Weeks 8–9 (Phases 6–7):** Tier 2 evaluation + cross-LOB aligner
- **Weeks 10–11:** Final report + 3-column comparison table + Streamlit demo

Deliverables: (1) LangGraph-compatible pipeline; (2) Neo4j graphs (flood / auto / SharedCore); (3) Evaluation (Tier 1 + Tier 2 + CTR/PTR); (4) Demo (Streamlit); (5) Report (10–15 pages).

References

[1] Y. Tiwari et al., "OntoRAG: Enhancing QA through automated ontology derivation," *arXiv:2506.00664*, 2025.

[2] J. Bai et al., "AutoSchemaKG: Autonomous KG construction through dynamic schema induction," *arXiv:2505.23628*, 2025.

[3] B. Zhang and H. Soh, "Extract, define, canonicalize: An LLM framework for KG construction," *EMNLP*, 2024.

[4] D. Edge et al., "From local to global: A Graph RAG approach," *arXiv:2404.16130*, 2024.

[5] S. A. Tartir and I. B. Arpinar, "Ontology evaluation and ranking using OntoQA," *Proc. Int'l Workshop on Semantic Computing and Applications*, 2007.

[6] FEMA, "Standard Flood Insurance Policy — General Property Form (F-123)," 2021. `https://www.fema.gov/sites/default/files/2020-05/F-123_SFIP_GeneralPropertyForm.pdf`

A Related Work Details

OntoRAG: Community detection for ontology induction; requires predefined relation types. We extend with fully typed extraction and cross-LOB transfer measurement.

AutoSchemaKG: Web-scale conceptualization (78K GPU hours). We adopt entity/event distinction but use local LLMs (no GPU cluster required).

EDC: Open IE with canonicalization; flat entity types, no hierarchy. We add Leiden-based type clustering and hierarchical induction.

GraphRAG: Community-based retrieval without formal ontology. We provide explicit schema enabling structured Cypher queries and cross-domain reasoning.

B Triple Quality and Typed Extraction Examples

C Cross-LOB Alignment Algorithm

```
Input:  flood ontology O_flood (Zone 3), auto ontology O_auto (Zone 3,
   independent run)
Output: SharedCore + FloodModule + AutoModule nodes in Neo4j

Embed all class names with MiniLM-L6-v2

For each auto class C_auto in O_auto:
    best_flood, score = max cosine similarity against O_flood

    If score >= 0.80:        # SHARED_CORE
        Create :SharedCore node (if not exists)
```

Quality	Triple	Reason
GOOD	`(Policy5123, Policy, HAS_COVERAGE_LIMIT, 250000, numeric, False, 0.98)`	Typed, specific, numeric
GOOD	`(Vehicle, VehicleType, NOT_INSURED, FloodDamage, CoverageType, True, 0.95)`	Negation correctly captured
GOOD	`(ProofOfLoss, Document, MUST_FILE_WITHIN_DAYS, 60, numeric, False, 0.97)`	Typed deadline
BAD	`(Policy, has, coverage, -, -, False, 0.45)`	Generic relation, no types, low conf
BAD	`(Thing, is, thing, -, -, False, 0.30)`	Self-referential, meaningless

Table 5: Key improvements over classic baseline: `subj_type`/`obj_type` fields guide ontology induction; `negated=True` enables NOT_INSURED triples (0% in baseline, target >50% recall); numeric values explicitly typed.

```
        C_auto -[:SUBCLASS_OF]-> SharedCore
        C_flood -[:SUBCLASS_OF]-> SharedCore

    Elif score >= 0.60:      # SUBCLASS
        C_auto -[:SUBCLASS_OF]-> C_flood  (in FloodModule)

    Else:                    # DOMAIN_SPECIFIC
        C_auto stays in AutoModule only

CTR = (#SharedCore_auto + #Subclass_auto) / |O_auto|
PTR = property overlap ratio
```

Concrete examples:

- Auto "Claim" ↔ Flood "Claim" (cosine 0.99) → **SharedCore** — enables cross-domain claim queries
- Auto "AutoPolicy" ↔ Flood "FloodPolicy" (cosine 0.78) → **Subclass** — both under PropertyPolicy
- Auto "VehicleType" ↔ best match "BuildingType" (cosine 0.42) → **DomainSpecific** — auto-only concept

D Baseline Limitations Detail

Configuration: LLMGraphTransformer (LangChain) + Llama 3.1 8B + direct Neo4j insertion, fixed 512-token chunks, PDF only (CSV excluded due to filter bug).

Observed issues:

- 20–30% entity duplication ("Policy 5123" vs. "policy #5123")

- Type inconsistency: same concept appears as "Policy", "InsurancePolicy", "flood policy"
- CSV chunks excluded: `.endswith(".pdf")` filter bug — 20 of 69 chunks missing
- No negation handling — "not covered" triples lost entirely
- No ontology structure — cannot answer "what are all subclasses of Coverage?"
- No cross-LOB capability — flood and auto produce isolated, unqueryable subgraphs

Note: All ∅ N/A cells in the Tier 2 final table are *not limitations of our evaluation*—they are the *finding*. The classic approach cannot be evaluated on those metrics because it never produces the artifacts those metrics require.

E SFIP Gold Standard Design

Why SFIP, not Riskine or FIBO:

- **Riskine** (github.com/riskine/ontology): commercial ontology not domain-matched to NFIP flood; no section-level annotations
- **FIBO**: 300+ classes, flood-specific concepts absent, designed for financial reporting not claims processing
- **SFIP gold standard** (hand-annotated from source): directly from the policy text we process; absence of a public NFIP ontology is *exactly the gap this project fills*; domain expert annotation from source is standard practice in ontology evaluation literature

35 classes from SFIP section headers:

- §I Coverage: `FloodCoverage`, `BuildingCoverage`, `ContentsCoverage`, `CoverageLimit`, `CoverageA`, `CoverageB`
- §II Definitions: `Flood`, `FloodEvent`, `Building`, `PostFIRMBuilding`, `Mudslide`, `SurfaceWater`, `Basement`
- §III/IV Exclusions: `Exclusion`, `EarthMovementExclusion`, `FinancialLossExclusion`, `LandExclusion`, `SewerBackupExclusion`
- §V Policy Terms: `Policy`, `WaitingPeriod`, `ICCCoverage`, `CoinsuranceProvision`, `LiberalizationClause`, `CancellationProvision`, `Endorsement`
- §VI Claims: `Claim`, `ProofOfLoss`, `LossAmount`, `BuildingDamageAmount`, `ContentsDamageAmount`, `SwornStatement`, `SubrogationRight`
- Parties: `PolicyHolder`, `Insurer`, `FloodZone`, `OccupancyType`, `ElevationCertificate`

14 properties: `hasCoverageLimit`, `hasDeductible`, `hasPremium`, `hasWaitingPeriod`, `hasFloodZone`, `hasOccupancyType`, `hasElevationCert`, `filedDate`, `lossDate`, `claimAmount`, `buildingDamage`, `contentsDamage`, `policyNumber`, `effectiveDate`

F Updated LLM Prompt Templates

Zone 2 Open IE (typed extraction):

```
You are extracting insurance knowledge triples.
Output ONLY valid JSON. Each triple MUST include subj_type and obj_type.

ALLOWED_ENTITY_TYPES: Policy, Claim, Coverage, FloodZone, Exclusion,
  Building, Contents, Deductible, ProofOfLoss, LossAmount, PolicyHolder,
  Insurer, WaitingPeriod, Endorsement, ElevationCert, OccupancyType,
  ICCCoverage, FloodEvent, Appurtenant, Basement

ALLOWED_RELATION_TYPES: HAS_COVERAGE, HAS_COVERAGE_LIMIT, EXCLUDED_FROM,
  NOT_INSURED, MUST_FILE_WITHIN_DAYS, SUBCLASS_OF, REQUIRES, FILED_AGAINST
      ,
  HAS_DEDUCTIBLE, HAS_PREMIUM, HAS_WAITING_PERIOD, APPLIES_TO, LOCATED_IN,
  OWNED_BY, AFFECTS, RESULTS_IN, SUBJECT_TO, HAS_PROPERTY, INSTANCE_OF,
  DETERMINES

EXAMPLES:
[GOOD] {"subject":"Vehicle","subj_type":"VehicleType",
        "relation":"NOT_INSURED","object":"FloodDamage",
        "obj_type":"CoverageType","negated":true,"confidence":0.95}
[GOOD] {"subject":"ProofOfLoss","subj_type":"Document",
        "relation":"MUST_FILE_WITHIN_DAYS","object":"60",
        "obj_type":"numeric","negated":false,"confidence":0.97}
[BAD]  {"subject":"Policy","relation":"has","object":"coverage","
   confidence":0.3}

Text: [chunk]
Output: [{"subject","subj_type","relation","object","obj_type","negated","
   confidence"}]
```

Zone 3 Class Naming:

```
Given these insurance entity types: {flood_policy, wind_policy,
   earthquake_policy}
Generate a single PascalCase abstract class name and one-sentence
   definition
that covers all member types.
Output JSON: {"name": "PropertyPolicy",
              "definition": "An insurance policy covering property..."}
```

Zone 3 Hierarchy Verification:

```
Class A: {name: "PropertyPolicy", properties: [coverage_limit, premium,
   flood_zone]}
Class B: {name: "FloodPolicy",    properties: [coverage_limit, premium,
   flood_zone,
                                                basement_coverage]}
Is B a subclass of A? (B has all properties of A plus domain-specific ones
   )
Answer: YES / NO / SIBLING
Confidence: 0.0-1.0
Reasoning: [brief explanation]
```